

72.39 Autómatas, teoría de lenguajes y compiladores

Stage III

Informe



Fecha de entrega: 14/06/2026

TABLA DE CONTENIDOS

1. Equipo	3
2. Repositorio	3
3. Introducción	3
4. Modelo Computacional	4
4.1. Dominio	4
4.2. Lenguaje	5
5. Implementación	9
5.1. Frontend	9
5.2. Backend	10
5.3. Adicionales	14
5.4. Dificultades Encontradas	14
5.5. Pruebas	15
6. Futuras Extensiones	16
7. Conclusiones	17
8. Referencias	17
9. Bibliografía	18

1. Equipo

Nombre	Apellido	Legajo	E-mail
Francisco	Costa	65.202	frcosta@itba.edu.ar
Franco	Branda	65.506	fbranda@itba.edu.ar
Román	Salerno	65.145	rsalerno@itba.edu.ar
Tiago	Heras	65.627	theras@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en el siguiente repositorio:

<https://github.com/franciscocostaa/PRL>

Commit hash: 7dbb00bb1c59714965e009cdda16d116fc92074b

3. Introducción

PRL, sigla de Pricing Rule Language, es un lenguaje específico de dominio (DSL) diseñado para definir reglas de precios de forma declarativa. El objetivo del lenguaje es representar campañas comerciales, descuentos, recargos, rechazos e invariantes de negocio sin acoplar esas reglas a un lenguaje de programación general ni a una implementación particular de backend.

Conceptualmente, PRL se ubica en el extremo declarativo del espectro de paradigmas: en lugar de describir cómo calcular un precio paso a paso (estilo imperativo), el programador especifica qué restricciones y políticas debe cumplir una operación comercial (las reglas e invariantes), y el compilador —y, a futuro, un motor resolutor— se encarga de validar esas especificaciones y de razonar sobre ellas. Esta lectura del lenguaje como un sistema basado en restricciones guió las decisiones de diseño de esta entrega final, en particular el rediseño de los bloques de simulación (ver §2.2 y §3.2).

El dominio elegido parte de un problema común en sistemas comerciales: las reglas de pricing suelen cambiar con frecuencia, dependen de campañas, segmentos de clientes, montos, cantidades y restricciones internas, y pueden quedar dispersas en código imperativo. PRL busca concentrar esa lógica en un artefacto legible, validable y

portable, que pueda ser procesado por un compilador y luego consumido por otros sistemas.

El proyecto se implementó sobre el framework base de Flex/Bison provisto para la materia. La entrega final incluye un frontend capaz de reconocer y parsear programas PRL, un AST propio para representar campañas, reglas y simulaciones, un backend con análisis semántico, una tabla de símbolos con scopes, un sistema de tipos que aprovecha schemas de entidades declarados por el usuario, y la generación de una salida JSON tanto para las campañas exportadas como para los problemas de satisfacción planteados por las simulaciones.

Este informe describe el modelo computacional del lenguaje, las decisiones de diseño del frontend y backend, las dificultades encontradas durante el desarrollo, las extensiones que quedaron fuera del alcance fundamental y las conclusiones del equipo.

4. MODELO COMPUTACIONAL

4.1. DOMINIO

El dominio modelado por PRL es el de reglas comerciales de pricing. Una regla de pricing determina qué acción debe aplicarse cuando se cumple una condición sobre los datos disponibles de una operación comercial. En este contexto, una operación puede involucrar entidades como cliente, producto u orden, y puede requerir aplicar descuentos porcentuales, descuentos fijos, recargos o rechazar la operación.

Las abstracciones principales del dominio son:

- Campaña (campaign): unidad principal del lenguaje. Agrupa entidades, invariantes y reglas bajo un nombre. Una campaña representa un conjunto coherente de políticas comerciales, por ejemplo PricingLatam, CyberMonday o RetailStandard.
- Entidades (entities): nombres simbólicos disponibles dentro de una campaña, opcionalmente acompañados de un schema de propiedades tipadas. Permiten declarar objetos del dominio sobre los cuales se escriben condiciones, por ejemplo client: Client { segment: string, tier: integer }.
- Propiedades (entidad.campo): atributos concretos de las entidades. Cuando la entidad declara un schema, cada acceso se valida y tipa contra él; ejemplos: client.segment, order.item_count, order.total_amount.

- Invariantes (invariants): condiciones que expresan restricciones globales de una campaña, por ejemplo `assert order.total_discount <= 40%`.
- Reglas (rules): unidades de decisión con nombre y prioridad. Cada regla tiene una condición `if` y una acción `then`.
- Prioridad (priority): número entero asociado a una regla. Permite representar orden relativo de evaluación o precedencia entre reglas.
- Acciones: efectos de negocio disponibles cuando se cumple una regla. El lenguaje soporta `discount`, `discount_fixed`, `surcharge` y `reject`.
- Ecuaciones: expresiones aritméticas escritas por el programador (+, -, , /) que permiten combinar propiedades y literales, por ejemplo `order.unit_price * order.item_count`.
- Simulación (simulation): planteo de un problema de satisfacción sobre un modelo: dadas ciertas restricciones (given) y un conjunto de restricciones esperadas (expect), se pregunta si existe (o no) una solución que las satisfaga simultáneamente.
- Exportación (export ... to ...): indica qué campaña debe materializarse como salida del compilador.

El valor de un DSL en este dominio es que permite separar la especificación de las reglas del código que las ejecuta o consume. En lugar de distribuir la lógica comercial entre múltiples servicios, clases o funciones, PRL permite escribir reglas en un formato cercano al negocio, someterlas a validaciones automáticas, y razonar sobre su satisfacibilidad antes de usarlas.

4.2. LENGUAJE

Un programa PRL está compuesto por cero o más declaraciones de campaña, cero o más simulaciones y cero o más instrucciones de exportación. La forma general implementada es la del Código 1.

```
campaign NombreCampaña {
  entities:
    entidad: Tipo { propiedad: tipo, ... }

  invariants:
    assert condición
```

```

rules:
  rule "Nombre_Regla" priority 1 {
    if condición
    then acción
  }
}

simulation "nombre" on NombreCampaña {
  given:
    condición
  expect:
    condición
    satisfiable
}

export NombreCampaña to json

```

Código 1: Estructura general de un programa PRL.

Las secciones entities, invariants y rules pueden aparecer dentro de una campaña, en cualquier orden y de forma opcional. La implementación actual permite campañas mínimas y también campañas con una o más reglas, invariantes y entidades.

Los elementos léxicos principales son:

- Palabras reservadas de estructura: campaign, entities, invariants, rules, rule, priority, if, then, assert, export, to, simulation, on, given, expect, satisfiable, unsatisfiable.
- Operadores lógicos y de pertenencia: and, or, not, in.
- Acciones reservadas: discount, discount_fixed, surcharge, reject.
- Identificadores: nombres de campañas, entidades, tipos y campos.
- Literales enteros: por ejemplo 100, 5000, 25.
- Literales string: por ejemplo "B2B" o "Operación rechazada".
- Literales porcentaje: un entero seguido de %, por ejemplo 25% o 40%.
- Comentarios: comentarios de línea con // y comentarios de bloque con / ... /.

Las condiciones soportadas incluyen comparaciones, pertenencia, composición lógica y ecuaciones aritméticas:

- Operadores relacionales: `==`, `!=`, `>`, `<`, `>=`, `<=`.
- Operadores aritméticos: `+`, `-`, `,`, `/`.
- Operadores lógicos: `and`, `or`, `not`.
- Operador de pertenencia: `in [ ... ]`.
- Paréntesis: permiten agrupar condiciones.

La precedencia definida en la gramática, de menor a mayor, es `or` < `and` < `not` < relacionales < aditivos (`+` `-`) < multiplicativos (`/`). De este modo, una ecuación como `client.tier * 2 + order.item_count >= 100` se interpreta como `((client.tier * 2) + order.item_count) >= 100`, reflejando la convención algebraica habitual y evitando ambigüedades en condiciones compuestas. La gramática resultante no presenta conflictos `shift/reduce` ni `reduce/reduce`.

El Código 2 muestra un ejemplo representativo del núcleo del lenguaje, con un schema de entidades, una invariante, una regla con una ecuación, una simulación de satisfacibilidad y una exportación.

```
campaign PricingLatam {
  entities:
    client: Client { segment: string, tier: integer }
    order: Order { item_count: integer, total_amount: integer, total_discount:
percentage }

  invariants:
    assert order.total_discount <= 40%

  rules:
    rule "Mayorista_B2B" priority 50 {
      if client.segment == "B2B" and order.item_count * 2 >= 100
      then discount(25%)
    }
}

simulation "Existe combo válido" on PricingLatam {
  given:
    client.segment == "B2B"
```

```

    order.item_count >= 100
  expect:
    order.total_amount > 0
  satisfiable
}

export PricingLatam to json

```

Código 2: Ejemplo representativo del núcleo de PRL.

El lenguaje se diseñó con una gramática libre de contexto que tiene como símbolo inicial `program`. A nivel de implementación, `program` deriva una lista de elementos de primer nivel (`campaign`, `simulation` o `export_stmt`). Cada campaña contiene un cuerpo compuesto por secciones opcionales de entidades, invariantes y reglas. Las reglas contienen condiciones y acciones. Las expresiones se reducen a accesos a campos, literales o ecuaciones aritméticas entre ambos, lo que mantiene el lenguaje acotado al dominio de reglas de pricing pero le da al programador libertad para escribir sus propias fórmulas.

SIMULACIÓN COMO SATISFACIBILIDAD

A diferencia del diseño original del Stage I —donde la simulación se concebía como una serie de validaciones individuales (`assertions`) sobre un caso concreto—, en esta entrega la simulación se replanteó como un problema de satisfacción de restricciones. Una simulación toma un modelo (una campaña), un conjunto de restricciones `given` que acotan el espacio de soluciones, y un conjunto de restricciones `expect` que una solución debe cumplir adicionalmente. La pregunta que plantea es: ¿existe una asignación de valores para las entidades del modelo que satisfaga simultáneamente todas las restricciones `given` y `expect`? La palabra clave `satisfiable` / `unsatisfiable` permite al autor expresar el resultado esperado de esa pregunta (Código 3).

```

simulation "Ventana imposible" on Bounded {
  given:
    order.total >= 100
  expect:
    order.total <= 10
  unsatisfiable
}

```

Código 3: Simulación que plantea (y espera) un sistema insatisfacible.

Esta visión es coherente con el carácter declarativo del lenguaje: el programador describe el modelo y las preguntas, no el algoritmo de búsqueda. El compilador valida que el problema esté bien formado (modelo existente, restricciones bien tipadas) y lo materializa como un artefacto que un motor resolutor externo puede consumir para buscar una solución o demostrar que no existe (ver §3.2 y §4).

5. IMPLEMENTACIÓN

5.1. FRONTEND

El frontend está compuesto por el análisis léxico, el análisis sintáctico y la construcción del AST. Su responsabilidad es transformar el texto fuente PRL en una representación estructurada que pueda ser validada por el backend.

El análisis léxico se implementó con Flex en `src/main/c/frontend/lexical-analysis/FlexPatterns.l`. Allí se definen los patrones para palabras reservadas, operadores relacionales y aritméticos, símbolos, identificadores, enteros y strings. El porcentaje se tokeniza como un entero seguido por el símbolo %, lo que luego permite construir un literal porcentual desde la gramática. El operador de división / no colisiona con los comentarios // y / porque Flex aplica la regla de coincidencia más larga. También se agregaron comentarios de línea y comentarios multilínea usando un estado exclusivo `MULTILINE_COMMENT`.

El análisis sintáctico se implementó con Bison en `src/main/c/frontend/syntactic-analysis/BisonGrammar.y`. La gramática define la estructura de programas, campañas, secciones, schemas de entidades, reglas, condiciones, expresiones aritméticas, simulaciones, acciones y exportaciones. El parser usa la modalidad push-parser de Bison (`%define api.push-pull push`), integrada con el scanner de Flex desde `Frontend.c`. La precedencia y asociatividad de los operadores lógicos, relacionales y aritméticos se declara explícitamente con `%left`, `%right` y `%nonassoc`, lo que permitió incorporar ecuaciones sin introducir conflictos en la gramática.

Durante el parseo se construye un AST específico para PRL, declarado en `src/main/c/frontend/syntactic-analysis/AbstractSyntaxTree.h`. Los nodos principales son:

- `ProgramNode`: raíz del programa, con una lista de elementos top-level.
- `TopLevelNode`: representa una campaña, una simulación o una exportación.

- CampaignNode: contiene nombre, entidades, invariantes y reglas.
- EntityDeclNode: representa una declaración entidad: Tipo con su lista opcional de propiedades.
- PropertyNode: representa una propiedad tipada nombre: tipo dentro del schema de una entidad.
- InvariantNode: contiene la condición de un assert.
- RuleNode: contiene nombre, prioridad, condición y acción.
- ConditionNode: representa condiciones lógicas, comparaciones, pertenencia in o expresiones booleanas.
- ExpressionNode: representa accesos entity.field, literales o ecuaciones aritméticas (con operandos izquierdo y derecho).
- ActionNode: representa discount, discount_fixed, surcharge o reject.
- SimulationNode: representa una simulación, con su modelo, sus restricciones given, sus restricciones expect y la expectativa de satisfacibilidad.
- ExportNode: representa export Campaña to formato.

Las acciones semánticas de Bison, ubicadas en BisonActions.c, encapsulan la creación y el agregado de nodos al AST. Esto permitió separar la forma gramatical de la representación interna del lenguaje.

Una decisión importante fue definir destructores de Bison para símbolos con memoria dinámica (incluyendo los nuevos nodos de simulación y las listas temporales de propiedades). Esto reduce pérdidas de memoria en escenarios donde el parser debe descartar símbolos por errores sintácticos. Además, el AST tiene destructores recursivos para liberar los nodos creados durante la compilación.

5.2. BACKEND

El backend se organiza en dos etapas: análisis semántico y generación de salida. Solo se ejecuta si el frontend logra construir correctamente el AST.

El flujo principal está en `src/main/c/EntryPoint.c`. Primero se inicializan los módulos del compilador, luego se ejecuta el análisis sintáctico. Si este análisis finaliza correctamente, se ejecuta el análisis semántico. Si el programa también supera esa etapa, finalmente se invoca el generador.

ANÁLISIS SEMÁNTICO

El análisis semántico está implementado en `src/main/c/backend/semantic-analysis/SemanticAnalyzer.c`. Su objetivo es rechazar programas que son sintácticamente válidos pero inconsistentes respecto del modelo de PRL.

La tabla de símbolos está implementada en `src/main/c/backend/semantic-analysis/SymbolTable.c` y `SymbolTable.h`. Se modeló como una pila de scopes. Cada scope contiene un arreglo dinámico de símbolos. Las búsquedas recorren la pila desde el scope más interno hacia el global, lo que permite representar visibilidad por contexto.

El análisis usa dos pasadas principales:

- Primera pasada: declara todas las campañas en el scope global. Esto permite que un export o una simulation puedan referenciar campañas declaradas antes o después en el archivo.
- Segunda pasada: valida el cuerpo de cada campaña, las referencias de cada exportación y la consistencia de cada simulación.

Las validaciones implementadas son:

- Unicidad de nombres de campaña.
- Unicidad de entidades dentro de una campaña.
- Unicidad de propiedades dentro del schema de una entidad.
- Unicidad de reglas dentro de una campaña.
- Existencia de la campaña referenciada por cada export y por cada simulation.
- Binding de accesos `entity.field` contra entidades declaradas.

- Verificación de que un acceso `entity.field` referencie una propiedad existente, cuando la entidad declara un schema.
- Chequeo de tipos en comparaciones, ecuaciones aritméticas y acciones.

El sistema de tipos interno define los tipos `integer`, `percentage`, `string`, `boolean`, `field` y `TYPE_ERROR`. Cuando una entidad declara un schema, sus campos dejan de ser dinámicos (`field`) y pasan a tener el tipo declarado por el programador (`integer`, `percentage`, `string`, etc.), de modo que el chequeo de tipos deja de estar `hardcodeado` y pasa a derivarse de la especificación del usuario. Si la entidad no declara propiedades, los accesos se siguen tratando como `field`, conservando la flexibilidad de los programas mínimos. De esta forma, una comparación como `client.segment >= 100` se rechaza cuando `segment` está declarado como `string`, mientras que el mismo acceso sin schema se acepta.

Las ecuaciones aritméticas se tipan numéricamente: el resultado de `+`, `-`, `*`, `/` es numérico si ambos operandos lo son, y se reporta un error si alguno es `string` o `boolean` (por ejemplo, `client.segment * 2`). El lenguaje acepta coerción numérica simple entre enteros y porcentajes en comparaciones.

Las acciones se validan con reglas específicas: `discount(...)` y `surcharge(...)` exigen un literal porcentaje, `discount_fixed(...)` exige un literal entero, y `reject(...)` exige un `string`, garantizado por la gramática.

Para las simulaciones, el análisis semántico verifica que el modelo referenciado exista y que cada restricción `given` y `expect` esté bien tipada contra el schema de ese modelo. Es decir, antes de que un eventual resolutor intente responder la pregunta de satisfacibilidad, el compilador garantiza que el problema esté bien formado.

Si se detecta al menos un error semántico, el compilador devuelve un estado de fallo y no ejecuta generación de código.

GENERACIÓN DE CÓDIGO

La generación está implementada en `src/main/c/backend/code-generation/Generator.c`. En esta etapa se recorre el AST y se serializan a JSON, por `stdout`, tanto las campañas exportadas como las simulaciones planteadas.

Cada exportación produce un objeto JSON con el nombre de la campaña, el formato indicado, la lista de entidades declaradas (con sus propiedades tipadas), la lista de invariantes y la lista de reglas (nombre, prioridad, condición y acción). La condición

de cada regla se renderiza como una cadena legible dentro del campo if, incluyendo las ecuaciones aritméticas. También se implementó escape de strings para emitir JSON válido frente a comillas, barras invertidas y caracteres de control.

Cada simulación produce un objeto JSON que describe el problema de satisfacción de restricciones: el modelo sobre el que corre, la lista de restricciones given, la lista de restricciones expect y la question planteada (satisfiable, unsatisfiable o solve). El Código 4 muestra una salida representativa.

```
[
{
  "simulation": "Existe combo válido",
  "model": "PricingLatam",
  "given": [ "client.segment == \"B2B\"", "order.item_count >= 100" ],
  "expect": [ "order.total_amount > 0" ],
  "question": "satisfiable"
},
{
  "campaign": "PricingLatam",
  "format": "json",
  "entities": [ { "name": "order", "type": "Order", "properties": [ { "name": "item_count",
"type": "integer" } ] } ],
  "invariants": [ "order.total_discount <= 40%" ],
  "rules": [
    { "name": "Mayorista_B2B", "priority": 50, "if": "client.segment == \"B2B\" and
order.item_count * 2 >= 100", "then": { "action": "discount", "value": "25%" } }
  ]
}
]
```

Código 4: Salida JSON con una simulación (como problema de satisfacibilidad) y una campaña exportada.

RUNTIME

El proyecto no implementa todavía el resolutor que busca soluciones concretas para las simulaciones. La decisión es consistente con el alcance de la entrega y con la guía de la cátedra de priorizar la parte fundamental del backend: el compilador valida un programa PRL, deriva el sistema de restricciones y lo materializa en un formato que un motor de satisfacción (por ejemplo, basado en un solver SMT o de programación por restricciones) podría consumir. La búsqueda efectiva de soluciones —o la prueba de insatisfacibilidad— queda como extensión natural descrita en §4.

5.3. ADICIONALES (OPCIONAL)

Además del núcleo mínimo del compilador, se implementaron varias decisiones y funcionalidades complementarias:

- Comentarios de línea y de bloque en el scanner.
- Operador de pertenencia in [...].
- Ecuaciones aritméticas definidas por el programador (+, -, *, /) con precedencia explícita.
- Schemas de entidades: propiedades tipadas declaradas por el usuario, que alimentan el chequeo de tipos en lugar de tenerlo hardcodeado.
- Bloques de simulación replanteados como problemas de satisfacción de restricciones, con given, expect y expectativa satisfiable / unsatisfiable.
- Precedencia explícita de operadores lógicos, relacionales y aritméticos.
- Acciones adicionales además de descuento porcentual: discount_fixed, surcharge y reject.
- Soporte para múltiples campañas, múltiples simulaciones y múltiples instrucciones export.
- Validación semántica de campañas, reglas, entidades y propiedades duplicadas, referencias desconocidas (exports y simulaciones) y tipos incorrectos.
- Escape de strings y emisión de simulaciones en la salida JSON generada.

5.4. DIFICULTADES ENCONTRADAS

Una dificultad importante fue administrar correctamente la memoria de los nodos del AST y de los valores semánticos creados durante el parseo. Al trabajar en C, cada transferencia de ownership entre Flex, Bison, las acciones semánticas y el AST debía quedar clara. Para reducir errores, se implementaron destructores recursivos para

los nodos y destructores de Bison para símbolos con memoria dinámica, incluyendo las listas temporales de expresiones (operador in) y de propiedades (schemas).

La incorporación de ecuaciones aritméticas exigió pensar la precedencia con cuidado. Como las comparaciones se construyen a nivel de condición (`expression rel_op expression`) y las ecuaciones a nivel de expresión, fue necesario declarar la precedencia de los operadores relacionales por debajo de la de los aritméticos para que el parser redujera primero las ecuaciones. El resultado es una gramática sin conflictos `shift/reduce` ni `reduce/reduce`.

Otro problema apareció con la condición in. La lista temporal de expresiones usada durante el parseo se transfiere al nodo `ConditionNode`. Si el `ownership` no se transfería de forma consistente, podían aparecer dobles liberaciones o leaks. La solución fue hacer que `createInConditionNode` tome `ownership` del arreglo de opciones y que la acción de Bison libere solo la estructura contenedora temporal. Se aplicó el mismo patrón a la lista temporal de propiedades de un schema.

También hubo que decidir cómo validar accesos `entity.field`. La solución adoptada es condicional: si la campaña declara entidades, se exige que los accesos referencien alguna de ellas; si además la entidad declara un schema, se exige que el campo exista y se usa su tipo declarado; si no declara entidades, el compilador permite esos accesos y los trata como campos dinámicos. Esto le da libertad al programador sin sacrificar la posibilidad de validaciones estrictas.

Finalmente, replantear la simulación como satisfacibilidad obligó a separar claramente dos responsabilidades: la del compilador (validar que el problema esté bien formado y materializarlo) y la de un eventual resolutor (buscar la solución). Mantener esa frontera permitió entregar una simulación con sentido declarativo aun sin implementar el motor de búsqueda.

5.5. PRUEBAS

El proyecto incluye un script de pruebas en `src/main/bash/test.sh`. El script compila todos los programas ubicados en `src/test/c/prl/accept/` esperando que sean aceptados, y todos los programas ubicados en `src/test/c/prl/reject/` esperando que sean rechazados. La entrega final incluye 18 casos de aceptación y 18 casos de rechazo, todos en verde.

Los casos de aceptación cubren, entre otros: campañas mínimas; reglas con descuento porcentual; reglas con reject; condiciones compuestas; operador in; prioridades intercaladas; comentarios mezclados; múltiples campañas; múltiples

exportaciones; acciones discount_fixed y surcharge; ecuaciones aritméticas; schemas de entidades tipados; y simulaciones satisfacibles e insatisfacibles.

Los casos de rechazo cubren, entre otros: campañas sin nombre; reglas sin then; acciones incompletas; llaves sin cerrar; caracteres inválidos; operadores relacionales faltantes; reglas fuera de campaña; prioridades sin valor; entidades no declaradas; campañas duplicadas; exportaciones a campañas inexistentes; reglas duplicadas; tipos incorrectos en acciones; acceso a una propiedad inexistente del schema; incompatibilidad de tipos derivada del schema; aritmética sobre operandos no numéricos; simulaciones sobre modelos inexistentes; y propiedades duplicadas.

Estas pruebas funcionan como validación de regresión del núcleo léxico, sintáctico y semántico del lenguaje.

6. FUTURAS EXTENSIONES

El diseño del lenguaje contempla funcionalidades que quedaron fuera del alcance fundamental de esta entrega. Se mantienen como futuras extensiones porque requieren una ampliación significativa del backend o la incorporación de un motor de resolución.

- Resolutor de simulaciones: el paso natural sobre la simulación ya implementada es construir el motor que efectivamente busca soluciones —o demuestra su inexistencia— para el sistema de restricciones generado. Esto implicaría traducir las condiciones PRL a la entrada de un solver SMT (por ejemplo, Z3) o de un sistema de programación por restricciones, y reportar un modelo concreto o unsatisfiable.
- Herencia de campañas: permitir una construcción como `extend CyberMonday from PricingLatam { ... }`, resolviendo la campaña base, copiando reglas heredadas, permitiendo overrides y modelando eliminación de reglas con construcciones como `drop rule`.
- Análisis estático `detect_shadowing`: detectar reglas inalcanzables por estar cubiertas por reglas de mayor prioridad, razonando sobre implicación o solapamiento entre condiciones.
- Schemas más ricos: extender el schema de entidades con restricciones de dominio por propiedad (rangos, enumeraciones) y con relaciones entre entidades, para enriquecer el espacio de búsqueda del resolutor.
- Validación de formatos de exportación: implementar explícitamente formatos alternativos como `sql` además de `json`.

- Mejoras de diagnóstico: agregar mensajes de error con más contexto, columna exacta y sugerencias para corregir programas inválidos.

7. CONCLUSIONES

El proyecto logró implementar un lenguaje específico de dominio para reglas de pricing con un flujo de compilación completo: análisis léxico, análisis sintáctico, construcción de AST, análisis semántico y generación de salida. PRL permite expresar campañas comerciales de forma declarativa, escribir ecuaciones propias, declarar el schema de cada entidad y plantear preguntas de satisfacibilidad sobre el modelo, obteniendo una representación JSON validada y adecuada para ser consumida por otros sistemas.

Durante el desarrollo se reforzó la importancia de separar responsabilidades entre frontend y backend. Flex se utilizó para reconocer lexemas y descartar comentarios o espacios. Bison se encargó de validar la estructura gramatical y construir el AST. El backend agregó validaciones que no podían resolverse solo con sintaxis, como duplicados, referencias inexistentes, tipos incorrectos y consistencia de las simulaciones.

La implementación de la tabla de símbolos como pila de scopes permitió modelar de forma simple la diferencia entre nombres globales, como campañas, y nombres locales a una campaña, como entidades. El sistema de tipos pasó de estar hardcodeado a derivarse de los schemas declarados por el programador, lo que le da más libertad y a la vez habilita chequeos más estrictos. Finalmente, replantear la simulación como un problema de satisfacción de restricciones alineó la construcción con el carácter declarativo del lenguaje y dejó una base clara para incorporar un resolutor.

El resultado final cumple con el alcance central del proyecto y lo extiende con construcciones que enriquecen el lenguaje. Las principales limitaciones restantes —la búsqueda efectiva de soluciones, la herencia de campañas y el análisis estático avanzado— quedan delimitadas como una continuación natural del trabajo.

8. REFERENCIAS

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

- GNU Bison Manual. Secciones sobre push parsers, precedencia, ubicaciones y destructores de símbolos. <https://www.gnu.org/software/bison/manual/>
- Flex Manual. Secciones sobre patrones, reglas y start conditions. <https://westes.github.io/flex/manual/>
- Repositorio del proyecto PRL: [git@github.com:franciscocostaa/PRL.git](https://github.com/franciscocostaa/PRL).

9. BIBLIOGRAFÍA

- Material de la cátedra sobre análisis léxico.
- Material de la cátedra sobre análisis sintáctico.
- Material de la cátedra sobre análisis semántico.
- Material de la cátedra sobre tabla de símbolos.
- Material de la cátedra sobre scopes.
- Material de la cátedra sobre sistema de tipos.
- Documento de Stage I del proyecto PRL: [doc/TLA-STAGE-I.pdf](#).
- Código fuente del proyecto, especialmente FlexPatterns.l, BisonGrammar.y, AbstractSyntaxTree.h, SemanticAnalyzer.c, SymbolTable.c, Generator.c y EntryPoint.c.